C# Data Structures & Algorithms — Syntax Cheat Sheet

> Quick reference for interviews and daily coding. Compatible with GitHub, Obsidian, Notion, VS Code Markdown Preview.

Table of Contents

1. Dictionary<TKey, TValue>
2. List
3. HashSet
4. SortedSet & SortedDictionary
5. Stack (LIFO)
6. Queue (FIFO)
7. LinkedList
8. PriorityQueue
9. Arrays
10. Common Algorithms
11. Big-O Quick Reference
12. LINQ Snippets
13. String Essentials

1. Dictionary<TKey, TValue>

Purpose: Key-value storage with fast lookup. Unordered. Average O(1) for add, search, delete.

```
// Initialization
var dict = new Dictionary<string, int>();
var dict2 = new Dictionary<string, int>
{
    { "apple", 1 },
    { "banana", 2 }
};

// Add / Update
```

```
dict["apple"] = 1;          // Add or update
dict.Add("banana", 2);      // Throws if key exists
dict["apple"] = 10;         // Update existing key

// ContainsKey  (KEY REFERENCE)
bool exists = dict.ContainsKey("apple");   // true / false

// ContainsValue
bool hasValue = dict.ContainsValue(10);

// Safe retrieval
if (dict.TryGetValue("banana", out int value))
{
    Console.WriteLine(value);   // 2
}

// Direct retrieval (throws KeyNotFoundException if missing)
int v = dict["apple"];

// Remove
dict.Remove("apple");

// Iterate
foreach (KeyValuePair<string, int> kvp in dict)
{
    Console.WriteLine($"{kvp.Key} = {kvp.Value}");
}

// Useful properties
dict.Count;
dict.Keys;
dict.Values;
dict.Clear();
```

Operation	Average	Worst
Add	O(1)	O(n)
Search (ContainsKey)	O(1)	O(n)
Remove	O(1)	O(n)

2. List

Purpose: Dynamic array. Ordered, index-based access.

```
// Initialization
var list = new List<int>();
var list2 = new List<int> { 1, 2, 3 };

// Add
list.Add(4);            // Append
list.Insert(0, 99);     // Insert at index

// Access
int first = list[0];
int count = list.Count;

// Search
bool has = list.Contains(4);        // O(n)
int idx  = list.IndexOf(2);         // -1 if not found

// Remove
list.Remove(4);         // Removes first occurrence
list.RemoveAt(0);       // Removes by index
list.Clear();           // Removes all

// Sort & Reverse
list.Sort();
list.Reverse();

// Iterate
foreach (int n in list) Console.WriteLine(n);

// Convert
int[] arr = list.ToArray();
```

3. HashSet

Purpose: Unique elements only. Fast membership check. Unordered.

```
var set = new HashSet<int>();
var set2 = new HashSet<int> { 1, 2, 3 };

// Add — returns false if already present
bool added = set.Add(4);

// Search
```

```
bool exists = set.Contains(2);   // O(1) average

// Remove
set.Remove(2);

// Set operations
set.UnionWith(new[] { 5, 6 });          // A ∪ B
set.IntersectWith(new[] { 1, 2 });      // A ∩ B
set.ExceptWith(new[] { 6 });            // A − B
set.SymmetricExceptWith(new[] { 1, 9 });// elements in one but not both

// Subset / superset checks
set.IsSubsetOf(otherSet);
set.IsSupersetOf(otherSet);

// Convert
var list = set.ToList();
```

4. SortedSet & SortedDictionary<TKey, TValue>

Purpose: Same as HashSet/Dictionary but keeps keys sorted. O(log n) operations.

```
// SortedSet
var sSet = new SortedSet<int> { 3, 1, 2 };   // {1, 2, 3}
sSet.Add(0);
int min = sSet.Min;
int max = sSet.Max;
var range = sSet.GetViewBetween(1, 2);       // {1, 2}

// SortedDictionary
var sDict = new SortedDictionary<string, int>
{
    { "banana", 2 },
    { "apple", 1 }
};
// Iterates in key order: apple, banana
foreach (var kvp in sDict) Console.WriteLine(kvp.Key);
```

5. Stack — LIFO

```
var stack = new Stack<string>();

stack.Push("first");
stack.Push("second");

string top  = stack.Peek();   // "second" (does not remove)
string pop  = stack.Pop();    // "second" (removes)
int    cnt  = stack.Count;
bool   has  = stack.Contains("first");

foreach (var item in stack) Console.WriteLine(item);
```

6. Queue — FIFO

```
var queue = new Queue<string>();

queue.Enqueue("first");
queue.Enqueue("second");

string front = queue.Peek();     // "first" (does not remove)
string deq   = queue.Dequeue();  // "first" (removes)
int    cnt   = queue.Count;
bool   has   = queue.Contains("second");

foreach (var item in queue) Console.WriteLine(item);
```

7. LinkedList

Purpose: Doubly linked list. O(1) insert/delete at known nodes.

```
var ll = new LinkedList<string>();

ll.AddLast("b");
ll.AddFirst("a");
ll.AddAfter(ll.First, "ab");
ll.AddBefore(ll.Last, "y");
```

```
LinkedListNode<string> node = ll.Find("b");

ll.Remove("b");
ll.RemoveFirst();
ll.RemoveLast();

foreach (var item in ll) Console.WriteLine(item);
```

8. PriorityQueue

Purpose: Min-heap by default. Use `Comparer` for max-heap or custom priority. (.NET 6+)

```
var pq = new PriorityQueue<string, int>();
pq.Enqueue("taskA", 3);
pq.Enqueue("taskB", 1);
pq.Enqueue("taskC", 2);

string next = pq.Peek();       // "taskB" (lowest priority number)
string dequeued = pq.Dequeue();// "taskB"

// Max-heap using custom comparer
var maxHeap = new PriorityQueue<string, int>(
    Comparer<int>.Create((a, b) => b.CompareTo(a)));
```

9. Arrays

```
int[] arr = new int[5];              // zeros
int[] arr2 = { 1, 2, 3, 4, 5 };

Array.Sort(arr2);
Array.Reverse(arr2);
int idx = Array.IndexOf(arr2, 3);

// 2D array
int[,] grid = new int[3, 3];
grid[0, 1] = 5;

// Jagged array
```

```
int[][] jag = new int[3][];
jag[0] = new int[] { 1, 2 };
```

10. Common Algorithms

Binary Search

```
int BinarySearch(int[] arr, int target)
{
    int lo = 0, hi = arr.Length - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

BFS (Graph / Grid)

```
var queue = new Queue<int>();
var visited = new HashSet<int>();
queue.Enqueue(start);
visited.Add(start);

while (queue.Count > 0)
{
    int node = queue.Dequeue();
    foreach (int next in adj[node])
    {
        if (!visited.Contains(next))
        {
            visited.Add(next);
            queue.Enqueue(next);
        }
    }
}
```

DFS (Recursive)

```
void Dfs(int node, HashSet<int> visited, List<int>[] adj)
{
    visited.Add(node);
    foreach (int next in adj[node])
    {
        if (!visited.Contains(next))
            Dfs(next, visited, adj);
    }
}
```

Two Pointers

```
int l = 0, r = arr.Length - 1;
while (l < r)
{
    // process arr[l], arr[r]
    l++; r--;
}
```

Sliding Window

```
int left = 0, sum = 0, best = 0;
for (int right = 0; right < arr.Length; right++)
{
    sum += arr[right];
    while (sum > target) { sum -= arr[left++]; }
    best = Math.Max(best, right - left + 1);
}
```

11. Big-O Quick Reference

Structure	Access	Search	Insert	Delete	Ordered?
Array	O(1)	O(n)	O(n)	O(n)	Yes

Structure	Access	Search	Insert	Delete	Ordered?
List	O(1)	O(n)	O(1)*	O(n)	Yes
Dictionary	—	O(1)*	O(1)*	O(1)*	No
HashSet	—	O(1)*	O(1)*	O(1)*	No
SortedDictionary	—	O(log n)	O(log n)	O(log n)	Yes
SortedSet	—	O(log n)	O(log n)	O(log n)	Yes
Stack	O(1) top	O(n)	O(1)	O(1)	LIFO
Queue	O(1) front	O(n)	O(1)	O(1)	FIFO
LinkedList	O(n)	O(n)	O(1)	O(1)	Yes
PriorityQueue	O(1) peek	O(n)	O(log n)	O(log n)	By priority

**Average case. Worst case O(n).*

12. LINQ Snippets

```
using System.Linq;

list.Max();
list.Min();
list.Sum();
list.Average();
list.Distinct().ToList();
list.OrderBy(x => x).ToList();
list.OrderByDescending(x => x).ToList();
list.Where(x => x > 5).ToList();
list.Any(x => x == 3);
list.All(x => x > 0);
list.First();
list.FirstOrDefault();
list.GroupBy(x => x % 2).ToList();
list.Select(x => x * 2).ToList();
```

13. String Essentials

```
string s = "hello";

s.Length;
s[0];                       // 'h'
s.Substring(1, 3);          // "ell"
s.IndexOf("l");             // 2
s.Contains("ell");          // true
s.ToUpper(); s.ToLower();
s.Trim();
s.Split(',');               // string[]
string.Join(",", arr);
s.Replace("l", "L");
s.StartsWith("he");
s.EndsWith("lo");

// StringBuilder — for repeated concatenation
var sb = new System.Text.StringBuilder();
sb.Append("a");
sb.AppendLine("b");
string result = sb.ToString();

// Char helpers
char.IsDigit('5');
char.IsLetter('a');
char.ToUpper('a');
```

End of Cheat Sheet